

TABLE DES MATIERES

PHASE 1 - INVENTAIRE TECHNIQUE COMPLET	3
1.1 Architecture réelle du projet	3
1.2 Arborescence des fichiers sources	3
1.3 Base de données - 9 tables	4
1.4 APIs exposées - 16 endpoints	4
PHASE 2 - MATRICE DE PREUVE BLUEPRINT VERS CODE	5
2.1 Noyau MoA (Mixture of Agents)	5
2.2 Pipeline DAG	6
2.3 Agent de Réflexion	6
2.4 Services d'infrastructure	7
2.5 Mesures de sécurité	7
2.6 Les 7 Agents produits	8
PHASE 3 - VÉRIFICATION DÉTAILLÉE DES ÉCARTS	9
Ecart N1 - MoA sans cascading	9
Ecart N2 - DAG non parallèle	9
Ecart N3 - Réflexion simulée	9
Ecart N4 - AST fix avec regex	9
Ecart N5 - RL Router simplifié	9
Ecart N6 - Cache flushdb	10
Ecart N7 - Sandbox Docker stub	10
Ecart N8 - ReDoS blacklist	10

Ecart N9 - Pas de SSE/WebSocket	10
Ecart N10 - Auth bugs (import/tier)	10
Ecart N11 - DELETE sans ownership	11
Ecart N12 - CSP incomplete	11
Ecart N13 - LLMRouter sans retry	11
Ecart N14 - Pas de DAG visualisation	11
Ecart N15 - Pas de streaming UI	11
Ecart N16 - uuid.randomUUID() bug	12
Ecart N17 - IDOR sur GET /projects/:id	12
PHASE 4 - AUDIT D'EXECUTION REELLE	12
4.1 Verifications statiques (code present)	12
4.2 Framework de test - ABSENT	13
PHASE 5 - RECHERCHE DES IMPLEMENTATIONS CACHEES	13
PHASE 6 - CLASSIFICATION DES ECARTS	13
6.1 Ecart reels (fonctionnalite n'existe pas)	13
6.2 Ecart d'implementation (existe mais differemment)	14
6.3 Ecart volontaires (choix de developpeur)	14
6.4 Ecart documentaires	14
PHASE 7 - SCORE DE CONFORMITE JUSTIFIE	15
7.1 Scores par domaine	15
7.2 Calcul du score global	15
PLAN DE CORRECTION PRIORISE	16

PHASE 1 - INVENTAIRE TECHNIQUE COMPLET

1.1 Architecture réelle du projet

Le projet AgentForge est organisé en monorepo pnpm + Turborepo avec 4 packages: **api** (serveur Hono), **web** (client React/Vite), **shared** (types/schemas/constants) et **sandbox** (environnement Docker). L'infrastructure repose sur PostgreSQL 16, Redis 7, et un déploiement Docker Compose avec CI/CD GitHub Actions.

Composant	Technologie	Version	Fichier
API Server	Hono + @hono/node-server	4.6.0	packages/api/
ORM	Drizzle ORM + postgres.js	0.36.0	packages/api/src/db/
Web Client	React + Vite + Tailwind v4	19.0 / 6.0 / 4.0	packages/web/
State Management	Zustand + persist	5.0.0	packages/web/src/lib/
Routing	React Router v7	7.1.0	packages/web/src/App.tsx
Database	PostgreSQL 16 Alpine	16	infra/docker/
Cache L2	Redis 7 Alpine	7	infra/docker/
Build System	Turborepo + pnpm	2.3.0 / 9.15.0	turbo.json
CI/CD	GitHub Actions	-	.github/workflows/
Sandbox	Docker (node:20-slim + python3)	20	packages/sandbox/

Tableau 1.1 - Stack technologique complète

1.2 Arborescence des fichiers sources

Le monorepo contient **71 fichiers** (excluant node_modules, .git, dist). L'API contient les services core (CodeGenerator, SuperAgent, ReflectionAgent) et les services infrastructure (LLMRouter, CacheManager, CostOptimizer, EventManager, SandboxManager). Le package shared définit 7 types, 9 schemas Zod, 14 constantes et 5 utilitaires.

Package	Fichiers	Lignes	Exports
api/core/	3 (Generator, Orchestrator, Reflection)	1,167	3 classes + 2 interfaces
api/services/	6 (LLMRouter, Cache, Cost, Event, Sandbox, index)	1,163	5 classes + 1 singleton
api/routes/	4 (agents, auth, projects, index)	586	4 routers, 16 endpoints
api/middleware/	6 (auth, error, rate, logger, security, index)	212	6 middlewares
api/db/	2 (schema, index)	143	9 tables + 4 enums
shared/	18 (types, schemas, constants, utils)	1,660	36 types + 14 schemas
web/	17 (pages, components, lib, styles)	1,856	8 pages + 5 components
sandbox/	3 (Dockerfile, execute.sh, warm-pool.sh)	104	Docker + scripts

infra/	2 (docker-compose, init-db)	98	4 services Docker
--------	-----------------------------	----	-------------------

Tableau 1.2 - Inventaire des fichiers par package

1.3 Base de donnees - 9 tables

Table	Colonnes	FK	Fichier:Ligne
users	9 (id, email, name, passwordHash, tier, avatarUrl, preferences, createdAt, updatedAt)	-	schema.ts:10
projects	9 (id, userId, name, description, agentId, status, config, files, timestamps)	users.id	schema.ts:27
generation_sessions	15 (id, projectId, userId, agentId, prompt, status, output, dagState, etc.)	2 FK	schema.ts:41
rl_training_data	9 (id, agentId, promptFeatures, selectedModel, strategy, quality, etc.)	-	schema.ts:61
error_recovery_log	8 (id, sessionId, errorType, errorMessage, fixLevel, attempt, success, etc.)	sessions.id	schema.ts:75
cost_tracking	9 (id, userId, sessionId, provider, model, tokensPrompt, tokensCompletion, etc.)	2 FK	schema.ts:88
analytics_events	5 (id, userId, event, properties, timestamp)	users.id	schema.ts:101
deployments	7 (id, projectId, platform, url, status, config, timestamps)	projects.id	schema.ts:110
refresh_tokens	6 (id, userId, tokenHash, expiresAt, isRevoked, createdAt)	users.id	schema.ts:122

Tableau 1.3 - Schema de base de donnees Drizzle

1.4 APIs exposees - 16 endpoints

#	Methode	Chemin	Auth	Rate Limit	Validation
1	GET	/api	Non	Non	-
2	POST	/api/auth/register	Non	Non	RegisterSchema
3	POST	/api/auth/login	Non	Non	LoginSchema
4	POST	/api/auth/refresh	Non	Non	AUCUNE
5	POST	/api/auth/logout	Non	Non	AUCUNE
6	GET	/api/agents	Non	Non	-
7	GET	/api/agents/:id	Oui	Non	-
8	POST	/api/agents/:id/execute	Oui	Oui	ExecuteAgentSchema
9	POST	/api/agents/:id/execute-sync	Oui	Oui	ExecuteAgentSchema
10	GET	/api/agents/:id/executions	Oui	Non	-

11	POST	/api/agents/:id/cancel/:eid	Oui	Non	-
12	GET	/api/projects	Oui	Non	-
13	POST	/api/projects	Oui	Non	CreateProjectSchema
14	GET	/api/projects/:id	Oui	Non	-
15	PATCH	/api/projects/:id	Oui	Non	UpdateProjectSchema
16	DELETE	/api/projects/:id	Oui	Non	-

Tableau 1.4 - Inventaire complet des endpoints API

PHASE 2 - MATRICE DE PREUVE BLUEPRINT VERS CODE

Chaque exigence du Blueprint est vérifiée avec la localisation exacte dans le code source. La colonne Statut utilise la classification obligatoire: CONFIRME (preuve directe), PARTIEL (implementation incomplete), ABSENT (non trouve), NON VERIFIE (impossible a determiner statiquement).

2.1 Noyau MoA (Mixture of Agents)

Exigence	Statut	Fichier	Fonction/Ligne	Preuve
3-round cascading (Propose/Critique/Refine)	CONFIRME	CodeGenerator.ts	generate() L61, buildProposeMessages L178, buildCritiqueMessages L191, buildRefineMessages L216	3 rounds sequentielles avec emitEvent par round
9 LLM providers	CONFIRME	llm.ts L5-115	LLM_PROVIDERS object	claude-3.7, gpt-4o, deepseek-r1, gemini-2.5-pro, gpt-o1, llama-3.3, qwen-2.5, mistral-large, gemini-2.0-flash
Appel parallele des LLMs	CONFIRME	CodeGenerator.ts L252	callAllProviders()	Promise.allSettled pour appels paralleles
Selection meilleur proposal	CONFIRME	CodeGenerator.ts L304	selectBestProposal()	Score de confiance + selection du max
Consensus detection	CONFIRME	CodeGenerator.ts L323	hasConsensus()	Threshold 0.85, min 2 proposals
Intelligent fusion	CONFIRME	CodeGenerator.ts L329	synthesizeCritiques()	Synthese des critiques multi-LLM
MAX_MOA_ROUNDS = 3	CONFIRME	CodeGenerator.ts L46	Constant	readonly MAX_MOA_ROUNDS = 3
MIN_PROPOSALS_FOR_CONSENSUS = 2	CONFIRME	CodeGenerator.ts L47	Constant	readonly MIN_PROPOSALS_FOR_CONSENSUS = 2

Tableau 2.1 - Preuves du Noyau MoA

2.2 Pipeline DAG

Exigence	Statut	Fichier	Fonction/Ligne	Preuve
Tri topologique	CONFIRMÉ	dag.ts L3-35	topologicalSort()	Algorithme Kahn avec niveaux
Validation DAG (cycles)	CONFIRMÉ	dag.ts L38-82	validateDAG()	DFS avec recursion stack
Execution parallele par niveau	CONFIRMÉ	SuperAgent.ts L117-155	execute() L117-155	topologicalSort + Promise.allSettled par niveau
7 DAG templates (1 par agent)	CONFIRMÉ	agents.ts L39-470	DEV/SLIDES/DOC/DATA/RECHERCHE/EMAIL/MARKETING	Chaque config a dagTemplate avec nodes + edges
Noeuds input/process/output	CONFIRMÉ	types/agent.ts L62-70	DAGNode type	type: input process output

Tableau 2.2 - Preuves du Pipeline DAG

2.3 Agent de Reflection

Exigence	Statut	Fichier	Fonction/Ligne	Preuve
6 criteres ponderes	CONFIRMÉ	agents.ts L29-38	DEFAULT_REFLECTION_CRITERIA	Code Quality 30%, Functionality 25%, Performance 15%, Security 15%, UX 10%, Testing 5%
Threshold 0.95	CONFIRMÉ	agents.ts (DEV config L57)	confidenceThreshold: 0.95	Configure par agent (DEV=0.95, SLIDES=0.90, etc.)
Evaluation LLM reelle (Gemini 2.0 Flash)	CONFIRMÉ	ReflectionAgent.ts L29-97	evaluate()	Appel LLMRouter.call("gemini-2.0-flash") avec prompt d'evaluation structure
Auto-Fix Niveau 1: Pattern	CONFIRMÉ	ReflectionAgent.ts L98-128	patternFix()	6 patterns regex (console.log, any, var, ==, debugger, eslint-disable)
Auto-Fix Niveau 2: AST	PARTIEL	ReflectionAgent.ts L131-215	astFix()	6 transformations structurelles (return types, arrow functions, try-catch, etc.) MAIS sans dependency ts-morph - utilise regex avancees, pas un vrai AST parser
Auto-Fix Niveau 3: LLM	CONFIRMÉ	ReflectionAgent.ts L217-260	llmFix()	Appel LLM avec feedback pour correction
Max 3 tentatives auto-fix	CONFIRMÉ	SuperAgent.ts L362	executeAutoFix()	Boucle avec maxAttempts=3

Tableau 2.3 - Preuves de l'Agent de Reflection

2.4 Services d'infrastructure

Exigence	Statut	Fichier	Preuve
LLMRouter: 9 providers, retry, backoff	CONFIRMÉ	LLMRouter.ts	Singleton, callOpenAI/Anthropic/Google/Compatible, DEFAULT_RETRY_ATTEMPTS=2, sleep() avec jitter
Cache L1 (Map, 5min) + L2 (Redis, 1h)	CONFIRMÉ	CacheManager.ts	l1Cache Map, L1_TTL=300, L2_TTL=3600, redis ioredis
SHA-256 keying	CONFIRMÉ	CacheManager.ts L173	generateKey() avec createHash("sha256")
LRU eviction	CONFIRMÉ	CacheManager.ts L184	evictL1IfNeeded(), L1_MAX_SIZE=1000
Namespace agentforge:cache:*	CONFIRMÉ	CacheManager.ts L29	CACHE_PREFIX = "agentforge:cache:"
SCAN+DEL au lieu de FLUSHDB	CONFIRMÉ	CacheManager.ts L130-153	redis.scanStream({match}) + redis.del()
CostOptimizer RL-inspired	PARTIEL	CostOptimizer.ts	Historique performance, scoring qualite/cout, LEARNING_RATE=0.1 MAIS pas de vrai RL (pas de Q-learning, pas de policy gradient) - heuristique adaptive seulement
SSE Event streaming	CONFIRMÉ	EventManager.ts + agents.ts L88-163	ReadableStream SSE, registerClient/subscribe/broadcastEvent
Sandbox Docker + E2B	PARTIEL	SandboxManager.ts	executeDocker() via child_process MAIS executeE2B() sans @e2b/sdk dependency - echouerait au runtime
Warm Pool script	PARTIEL	warm-pool.sh	Script shell fonctionnel MAIS non integre dans le code TypeScript - SandboxManager ne l'appelle pas
Pre-compiled regex blacklist	CONFIRMÉ	SandboxManager.ts L15-32	17 RegExp pre-compilees en constante module
Cloudflare deployment	ABSENT	-	Aucun SDK @cloudflare, aucun wrangler, aucune integration Cloudflare Workers/Pages dans le code
OpenTelemetry / Monitoring	ABSENT	-	Aucune dependency @opentelemetry, aucun prom-client, aucune metric exportee

Tableau 2.4 - Preuves des Services d'infrastructure

2.5 Mesures de securite

Exigence	Statut	Fichier	Preuve
JWT HS256 Auth	CONFIRMÉ	auth.ts middleware L1-37	verify(token, JWT_SECRET, "HS256"), 15min access + 7d refresh
Zod validation (endpoints)	PARTIEL	agents.ts, projects.ts	ExecuteAgentSchema + CreateProjectSchema + RegisterSchema MAIS /auth/refresh et /auth/logout sans zValidator

Rate Limiting Redis	CONFIRME	rateLimiter.ts	3 tiers (free=10, pro=60, enterprise=300/min), X-RateLimit-Remaining header
CORS configurable	CONFIRME	index.ts L17-25	hono cors() avec origins, methods, credentials, maxAge=86400
XSS Sanitization	CONFIRME	validation.ts L11-18	sanitizeString() escape <, >, ", ', /
CSP complet	PARTIEL	securityHeaders.ts L38-52	11 directives CSP MAIS unsafe-eval + unsafe-inline affaiblissent la protection XSS
HSTS (production)	CONFIRME	securityHeaders.ts L22	max-age=63072000; includeSubDomains; preload
COOP/CORP/COEP	CONFIRME	securityHeaders.ts L27-29	Cross-Origin-Opener/Resource/Embedder-Policy
bcrypt password hashing	CONFIRME	auth.ts L34	bcrypt.hash(data.password, 12)
Refresh token DB storage	CONFIRME	auth.ts L38-48	SHA-256 hash stocke en DB, isRevoked flag
Parameterized queries (Drizzle)	CONFIRME	Tous les routes	Drizzle ORM genere des requetes parametrees automatiquement
IDOR protection	PARTIEL	projects.ts	DELETE + PATCH verifient userId MAIS GET /:id ne verifie PAS userId - IDOR sur lecture
Refresh token tid verification	ABSENT	auth.ts L139-144	Verifie userId + isRevoked mais PAS le tid specifique - tout token non-revoque pour l'utilisateur passe

Tableau 2.5 - Preuves des Mesures de securite

2.6 Les 7 Agents produit

Agent	LLMs	Seuil	DAG	Sandbox	Statut
DEV	3P + 2S + 2F = 7	95%	6 nodes, 5 levels	Oui	CONFIRME
SLIDES	2P + 1S + 1F = 4	90%	5 nodes, 5 levels	Non	CONFIRME
DOC	2P + 1S + 1F = 4	90%	5 nodes, 5 levels	Non	CONFIRME
DATA	2P + 1S + 1F = 4	90%	5 nodes, 5 levels	Oui	CONFIRME
RECHERCHE	2P + 1S + 1F = 4	92%	5 nodes, 5 levels	Non	CONFIRME
EMAIL	2P + 1S + 1F = 4	88%	4 nodes, 4 levels	Non	CONFIRME
MARKETING	2P + 1S + 1F = 4	88%	5 nodes, 5 levels	Non	CONFIRME

Tableau 2.6 - Configuration des 7 Agents (P=Primary, S=Secondary, F=Fallback)

PHASE 3 - VERIFICATION DETAILLEE DES ECARTS

Ecart N1 - MoA sans cascading

Statut: CONFIRME | **Confiance:** 100% | **Impact:** Fonctionnel

Fichier: CodeGenerator.ts L61-95

Preuve technique: 3 rounds Propose/Critique/Refine implementees avec buildProposeMessages, buildCritiqueMessages, buildRefineMessages. Chaque round appelle les LLMs et accumule les resultats.

VERDICT: Confirme corrige avec preuve technique directe.

Ecart N2 - DAG non parallele

Statut: CONFIRME | **Confiance:** 100% | **Impact:** Performance

Fichier: SuperAgent.ts L117-155

Preuve technique: topologicalSort() + Promise.allSettled par niveau. Les noeuds du meme niveau executent en parallele.

VERDICT: Confirme corrige avec preuve technique directe.

Ecart N3 - Reflection simulee

Statut: CONFIRME | **Confiance:** 100% | **Impact:** Fonctionnel

Fichier: ReflectionAgent.ts L29-97

Preuve technique: Appel reel a Gemini 2.0 Flash via LLMRouter.call(). Prompt d'evaluation structure avec 6 criteres.

VERDICT: Confirme corrige avec preuve technique directe.

Ecart N4 - AST fix avec regex

Statut: PARTIEL | **Confiance:** 75% | **Impact:** Fonctionnel

Fichier: ReflectionAgent.ts L131-215

Preuve technique: 6 transformations structurelles (return types, arrow functions, try-catch, semicolons, exports, missing params) MAIS sans ts-morph dependency. Utilise des regex avancees, pas un vrai parseur AST. Le code est fonctionnel pour les cas courants mais peut echouer sur du code complexe.

VERDICT: Partiellement corrige - ecart d'implementation subsistant.

Ecart N5 - RL Router simpliste

Statut: PARTIEL | **Confiance:** 60% | **Impact:** Performance/Scalabilite

Fichier: CostOptimizer.ts

Preuve technique: Historique des performances, scoring qualite/cout/latence, LEARNING_RATE=0.1, exponential moving average. C'est une heuristique adaptive, pas un vrai RL (pas de Q-table, pas de policy gradient, pas d'exploration/exploitation).

VERDICT: Partiellement corrige - ecart d'implementation subsistant.

Ecart N6 - Cache flushdb

Statut: CONFIRME | **Confiance:** 100% | **Impact:** Securite

Fichier: CacheManager.ts L130-153

Preuve technique: Remplace par SCAN+DEL avec namespace agentforge:cache:*. Seules les cle du namespace sont supprimees.

VERDICT: Confirme corrige avec preuve technique directe.

Ecart N7 - Sandbox Docker stub

Statut: PARTIEL | **Confiance:** 65% | **Impact:** Fonctionnel

Fichier: SandboxManager.ts L110-250

Preuve technique: executeDocker() via child_process implemente avec resource limits. MAIS executeE2B() sans @e2b/sdk - echouerait au runtime. Warm pool script existe mais non integre dans le code TS.

VERDICT: Partiellement corrige - ecart d'implementation subsistant.

Ecart N8 - ReDoS blacklist

Statut: CONFIRME | **Confiance:** 100% | **Impact:** Securite

Fichier: SandboxManager.ts L15-32

Preuve technique: 17 RegExp pre-compilees en constante module. Pas de new RegExp() en boucle.

VERDICT: Confirme corrige avec preuve technique directe.

Ecart N9 - Pas de SSE/WebSocket

Statut: CONFIRME | **Confiance:** 100% | **Impact:** Fonctionnel

Fichier: EventManager.ts + agents.ts L88-163

Preuve technique: SSE via ReadableStream, registerClient/subscribe/broadcastEvent. SSE stream retourne comme Response.

VERDICT: Confirme corrige avec preuve technique directe.

Ecart N10 - Auth bugs (import/tier)

Statut: CONFIRME | **Confiance:** 100% | **Impact:** Securite

Fichier: auth.ts

Preuve technique: Imports au top, tier recupere de la DB (user.tier), refresh token hashe en SHA-256 et stocke en DB.

VERDICT: Confirme corrige avec preuve technique directe.

Ecart N11 - DELETE sans ownership

Statut: CONFIRME | **Confiance:** 95% | **Impact:** Securite

Fichier: projects.ts L83-100

Preuve technique: DELETE verifie eq(projects.userId, userId). PATCH aussi.

VERDICT: Confirme corrige avec preuve technique directe.

Ecart N12 - CSP incomplete

Statut: PARTIEL | **Confiance:** 80% | **Impact:** Securite

Fichier: securityHeaders.ts L38-52

Preuve technique: 11 directives CSP + HSTS + COOP/CORP/COEP + Permissions-Policy. MAIS unsafe-eval et unsafe-inline dans script-src/style-src affaiblissent la protection.

VERDICT: Partiellement corrige - ecart d'implementation subsistant.

Ecart N13 - LLMRouter sans retry

Statut: CONFIRME | **Confiance:** 100% | **Impact:** Performance

Fichier: LLMRouter.ts L45-115

Preuve technique: DEFAULT_RETRY_ATTEMPTS=2, DEFAULT_RETRY_DELAY=1000, MAX_RETRY_DELAY=10000, sleep() avec exponential backoff.

VERDICT: Confirme corrige avec preuve technique directe.

Ecart N14 - Pas de DAG visualisation

Statut: CONFIRME | **Confiance:** 100% | **Impact:** Fonctionnel

Fichier: Execute.tsx L451

Preuve technique: DAG visualise par niveau avec statut (pending/running/completed/failed), MoA badges, live events feed.

VERDICT: Confirme corrige avec preuve technique directe.

Ecart N15 - Pas de streaming UI

Statut: CONFIRME | **Confiance:** 100% | **Impact:** Fonctionnel

Fichier: Execute.tsx + api.ts

Preuve technique: executeAgentStream() via fetch ReadableStream, SSE parsing, real-time DAG status updates.

VERDICT: Confirme corrige avec preuve technique directe.

Ecart N16 - uuid.randomUUID() bug

Statut: ABSENT | Confiance: 0% | Impact: Fonctionnel

Fichier: agents.ts L86

Preuve technique: Ligne 86 utilise uuid.randomUUID() qui n'existe pas dans le package uuid. Devrait etre crypto.randomUUID() ou uuidv4(). Ligne 184 utilise correctement crypto.randomUUID(). Bug encore present.

VERDICT: Non corrige - ecart reel persistant.

Ecart N17 - IDOR sur GET /projects/:id

Statut: ABSENT | Confiance: 0% | Impact: Securite

Fichier: projects.ts L37-46

Preuve technique: GET /:id ne verifie PAS userId. N'importe quel utilisateur authentifie peut lire n'importe quel projet par ID.

VERDICT: Non corrige - ecart reel persistant.

PHASE 4 - AUDIT D'EXECUTION REELLE

4.1 Verifications statiques (code present)

Fonctionnalite	Existe	Fonctionne	Demonstrable	Risque runtime
API Hono + routes	CONFIRME	PARTIEL	PARTIEL	uuid.randomUUID() bug L86
Auth JWT + bcrypt	CONFIRME	PARTIEL	PARTIEL	Refresh tid non verifie
MoA cascading	CONFIRME	PARTIEL	PARTIEL	Depend des cles API LLM
DAG parallel execution	CONFIRME	CONFIRME	CONFIRME	Topological sort + Promise.allSettled
Reflection (LLM eval)	CONFIRME	PARTIEL	PARTIEL	Depend GOOGLE_API_KEY
Auto-Fix 3 niveaux	CONFIRME	PARTIEL	PARTIEL	AST fix sans ts-morph
Cache L1+L2	CONFIRME	PARTIEL	PARTIEL	Depend REDIS_URL
Sandbox Docker	CONFIRME	PARTIEL	PARTIEL	E2B echouerait, Docker OK si daemon present
SSE streaming	CONFIRME	CONFIRME	CONFIRME	ReadableStream standard
Rate limiting	CONFIRME	PARTIEL	PARTIEL	Fallback open si Redis down

RBAC (tiers)	PARTIEL	PARTIEL	PARTIEL	Pas de middleware RBAC - tiers dans JWT seulement
Tests unitaires	ABSENT	ABSENT	ABSENT	Aucun test, aucun framework de test
Cloudflare deploy	ABSENT	ABSENT	ABSENT	Aucun SDK/integration

Tableau 4.1 - Audit d'exécution réelle

4.2 Framework de test - ABSENT

Aucun framework de test (vitest, jest, mocha) n'est installé dans les dépendances. Le script "test" dans le root package.json délègue à "turbo test" mais aucun sous-package ne définit un script test fonctionnel. La CI GitHub Actions exécute "pnpm test" qui échouerait en l'absence de tests. Il n'existe aucun fichier .test.ts ou .spec.ts dans tout le projet.

PHASE 5 - RECHERCHE DES IMPLEMENTATIONS CACHEES

Recherche exhaustive effectuée dans tous les répertoires, services, scripts, packages, configurations, modules partagés et dépendances internes. Résultats:

Dependency/Feature	Recherche	Résultat
ts-morph / TypeScript Compiler API	rg ts-morph typescript.*compiler babel swc	NON TROUVE - AST fix utilise regex avancées
@e2b/sdk	rg e2b @e2b	NON TROUVE - executeE2B() échouerait au runtime
@cloudflare/workers / wrangler	rg cloudflare @cloudflare wrangler	NON TROUVE - Pas d'intégration Cloudflare
@opentelemetry / prom-client	rg opentelemetry prom-client metrics	NON TROUVE - Pas de monitoring/metrics
vitest / jest / mocha	rg vitest jest mocha	NON TROUVE - Pas de framework de test
warmPool / WarmPool	rg warmPool warm.pool WarmPool	Script shell uniquement - non intégré au code TS
executeDAG function	rg executeDAG	NON TROUVE - DAG exécute dans SuperAgent directement
WebSocket / ws / socket.io	rg websocket socket.io ws:	NON TROUVE - SSE uniquement (pas de WebSocket)

Tableau 5.1 - Recherche exhaustive des implémentations cachées

PHASE 6 - CLASSIFICATION DES ECARTS

6.1 Ecart réel (fonctionnalité n'existe pas)

Cloudflare deployment [Critique]: Aucun SDK @cloudflare, aucun wrangler, aucune route de déploiement vers Cloudflare. Le blueprint spécifie cloudflare comme deploymentTarget pour DEV.

Tests unitaires [Elevee]: Zero fichier de test, zero framework de test installé. La CI exécute "pnpm test" qui ne produit rien.

Monitoring/OpenTelemetry [Elevee]: Aucune telemetrie, aucune metric, aucun dashboard. Impossibilité de monitorer la production.

RBAC middleware [Moyenne]: Pas de middleware de controle d'accès basé sur les rôles. Le tier est dans le JWT mais pas vérifié par un middleware dédié.

Refresh token tid verification [Elevee]: Le endpoint /auth/refresh accepte n'importe quel token non-revoque pour l'utilisateur, pas seulement le token spécifique.

IDOR sur GET /projects/:id [Critique]: La lecture d'un projet ne vérifie pas l'appartenance à l'utilisateur.

E2B SDK [Moyenne]: executeE2B() est implémenté mais sans la dependency @e2b/sdk - échec runtime garanti.

6.2 Ecart d'implémentation (existe mais différemment)

AST Auto-Fix [Moyenne]: Le blueprint spécifie ts-morph pour les transformations AST. L'implémentation utilise des regex avancées avec 6 transformations structurelles. Fonctionnel pour les cas courants, mais pas un vrai parseur AST.

RL Cost Optimizer [Moyenne]: Le blueprint décrit un router RL avec Q-learning. L'implémentation est une heuristique adaptative avec historique et exponential moving average. Pas de vrai apprentissage par renforcement.

Warm Pool [Faible]: Le script warm-pool.sh existe et est fonctionnel mais n'est pas intégré dans SandboxManager.ts. L'utilisateur doit le lancer manuellement.

CSP unsafe-eval/inline [Faible]: Le blueprint spécifie une CSP stricte. L'implémentation inclut unsafe-eval et unsafe-inline pour des raisons de compatibilité framework.

6.3 Ecart volontaires (choix de développeur)

WebSocket [Faible]: Le blueprint mentionne SSE et WebSocket. Seul SSE est implémenté - choix justifié par la simplicité et la compatibilité HTTP/2.

Rate limiter fallback open [Faible]: Quand Redis est indisponible, le rate limiter laisse passer les requêtes. Choix délibéré pour la résilience.

Projects page UI [Moyenne]: La page Projects.tsx est un placeholder vide. Le CRUD existe en API mais pas dans l'interface.

6.4 Ecart documentaires

docs/architecture-guide.md [Faible]: Le guide documente un "RL routing algorithm" mais l'implémentation est une heuristique, pas du RL. Le document est partiellement

obsolete.

API Reference doc [Moyenne]: Ne documente pas le endpoint SSE /execute (retourne un stream, pas du JSON). La doc est incomplete pour le streaming.

PHASE 7 - SCORE DE CONFORMITE JUSTIFIE

7.1 Scores par domaine

Domaine	Score	Justification
Architecture (Monorepo + Kernel + 7 Agents)	18/20	Architecture "1 Kernel, 7 Configurations" completement implementee. MoA 3-round cascading, DAG parallele, 7 agents avec DAG templates. -2: Cloudflare deployment cible absent.
MoA Kernel (9 LLMs, Cascading, Fusion)	17/20	3-round Propose/Critique/Refine, 9 providers, Promise.allSettled, consensus detection. -3: Pas de strategie weighted_vote dynamique.
DAG Pipeline (Topo sort + Parallele)	18/20	Topological sort + validateDAG + Promise.allSettled par niveau. 7 templates DAG. -2: Pas de conditional edges (edge.condition pas utilise).
Reflection + Auto-Fix	14/20	6 criteres ponderes, threshold 0.95, evaluation LLM reelle, 3-level fix. -6: AST fix sans ts-morph, pas de validation reelle du code genere.
RL Router / Cost Optimizer	10/20	Heuristique adaptive avec historique. -10: Pas de vrai RL (pas de Q-learning, pas d'exploration/exploitation), scoring statique.
Cache L1+L2	18/20	L1 Map 5min + L2 Redis 1h, SHA-256, LRU, namespace, SCAN+DEL. -2: Pas de cache warming, pas de stats L2.
Sandbox	12/20	Docker + blacklist + resource limits. -8: E2B non fonctionnel, warm pool non integre, pas de reutilisation de conteneurs.
Securite	14/20	JWT+bcrypt+Zod+rate limit+CSP+HSTS+COOP/CORP/COEP+XSS. -6: IDOR sur GET, refresh tid non verifie, unsafe-eval/inline.
SSE Streaming + UI	16/20	SSE ReadableStream, DAG visualisation, MoA badges, live events. -4: Pas de WebSocket, Projects page placeholder.
Tests	0/20	Zero test, zero framework, zero couverture. Le script "pnpm test" ne produit rien.
Monitoring/Observabilite	2/20	Health endpoint avec memoire/uptime. -18: Pas de OpenTelemetry, pas de metrics, pas de tracing distribue.
Deploiement Cloudflare	0/20	Aucune integration Cloudflare. Le deploymentTarget est defini mais non implemente.

Tableau 7.1 - Scores de conformite par domaine

7.2 Calcul du score global

Calcul pondere par importance relative:

Domaine	Poids	Score	Pondere
Architecture	15%	18/20	2.70
MoA Kernel	15%	17/20	2.55

DAG Pipeline	10%	18/20	1.80
Reflection + Auto-Fix	10%	14/20	1.40
RL Router	5%	10/20	0.50
Cache	5%	18/20	0.90
Sandbox	5%	12/20	0.60
Securite	10%	14/20	1.40
SSE + UI	5%	16/20	0.80
Tests	10%	0/20	0.00
Monitoring	5%	2/20	0.10
Deploiement Cloudflare	5%	0/20	0.00
TOTAL	100%	-	12.75/20

Tableau 7.2 - Calcul du score global pondere

SCORE GLOBAL DE CONFORMITE: 12.75 / 20 (63.75%)

Ce score reflète une implementation fonctionnelle du noyau architectural (MoA + DAG + Reflection) avec des lacunes significatives dans les domaines transversaux (tests, monitoring, deploiement Cloudflare) et des imperfections de securite (IDOR, refresh token). Les composants centraux du blueprint sont presents et fonctionnels; les ecart se concentrent sur la qualite de production et la robustesse operationnelle.

PLAN DE CORRECTION PRIORISE

#	Ecart	Priorite	Effort	Action
1	IDOR GET /projects/:id	CRITIQUE	1h	Ajouter eq(projects.userId, userId) dans le WHERE du SELECT
2	uuid.randomUUID() bug	CRITIQUE	5min	Remplacer par crypto.randomUUID() en L86 de agents.ts
3	Refresh tid verification	ELEVEE	2h	Verifier le hash tid du refresh token specifique en DB
4	Zod validation /auth/refres h+logout	ELEVEE	30min	Ajouter zValidator avec schema Zod pour les 2 endpoints
5	Tests unitaires + framework	ELEVEE	2-3j	Installer vitest, ecrire tests pour services core + routes API
6	E2B SDK ou supprimer executeE2B()	MOYENNE	2h	Installer @e2b/sdk ou marquer executeE2B() comme TODO
7	AST Fix avec ts-morph	MOYENNE	1j	Installer ts-morph, reimplementer les 6 transformations avec un vrai AST

8	RBAC middleware	MOYENNE	3h	Creer requireTier("pro") middleware pour les routes premium
9	Monitoring OpenTelemetry	MOYENNE	1j	Installer @opentelemetry/api + sdk, instrumenter LLMRouter + Cache
10	Cloudflare deployment	MOYENNE	1-2j	Ajouter wrangler + @cloudflare/workers, adapter le Dockerfile
11	Warm Pool integration	FAIBLE	2h	Integrer warm-pool.sh dans SandboxManager avec pre-warming au demarrage
12	CSP sans unsafe-eval	FAIBLE	3h	Nonce-based CSP pour scripts, hash-based pour styles
13	Projects page UI	FAIBLE	4h	Implementer CRUD Projects avec liste, creation, detail, suppression

Tableau 8.1 - Plan de correction priorise